Extreme TypeScript: Mastering Recursion and Inference via Type-Level Arithmetic

CKEditor

Ondřej Chrastina

Developer Advocate

CKEditor

ondrej.chrastina.dev

How it all started ?

Jirka 💜 Haskell

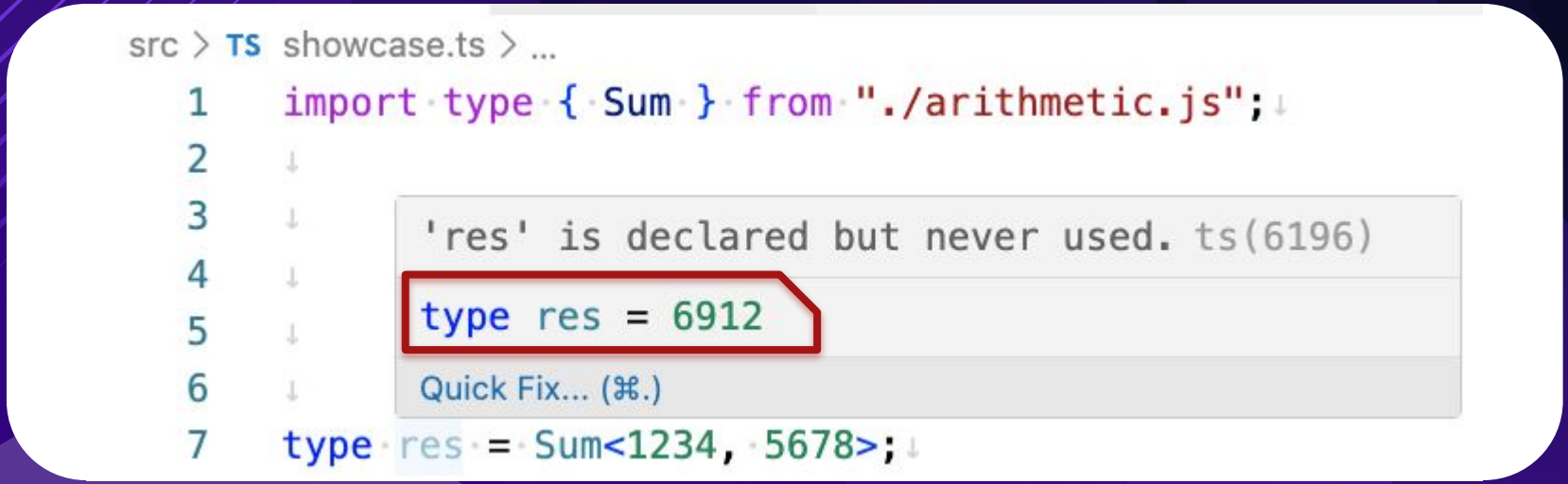
https://github.com/JiriLojda

Review: “ What is that? ”

yargs.ts

```
export type RegisterCommand = <Result, InitialParams extends LogOptions =
LogOptions>(
  obj: Readonly<{
    command: <CommandParams extends InitialParams = InitialParams>(
      cmdModule: StandaloneCommandModule<InitialParams, CommandParams>,
    ) => Result;
  }>,
) => Result;
```

Goal: READ complex types



Building blocks
Sum 2 DIGITS - the trick!

Literal definition

```
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9;
```

Map

```
type DigitToTupleMap = {
  0: [];
  1: [0];
  2: [0, 0];
  3: [0, 0, 0];
  // ... up to 9
  9: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0];
};
```

```
type res = DigitToTuple[2] // [0,0]
```

Generics & Spread operator

```
type ConcatTuples<A extends Digit, B extends Digit> =
  [...DigitToTuple[A], ...DigitToTuple[B]];

type res = ConcatTuples<2, 3>; // [0,0,0,0,0]
```

Length

```
// 5
type res = [0, 0, 0, 0, 0]['length']
```

First TS calculator!

```
type A = DigitToTuple[3]; // [0,0,0]
type B = DigitToTuple[5]; // [0,0,0,0,0]
type LengthA = A['length'] // 3
type ABConcat = [...A, ...B] // [0,0,0,0,0,0,0,0]
type SumAB = ABConcat['length'] // 8
```

Building blocks
Sum 2 numbers WITHOUT CARRY

Number to String (via Template Literal Types)

```ts
type NumberToString<N extends number> = `${N}`;

type A = NumberToString<3>;   // "3"
type B = NumberToString<123>; // "123"
type C = `${42}`;             // "42"
```

```ts
type StrToTuple<
  T extends string,
  Accum extends readonly string[] = []
> = T extends `${infer Fst}${infer Rest}`
  ? StrToTuple<Rest, [...Accum, Fst]>
  : Accum;

// ["1", "2", "3"]
type res = StrToTuple<"123">
```

```ts
type StrToTupleMap = {
  '0': [];
  '1': [0];
  '2': [0, 0];
  '3': [0, 0, 0];
  // ... up to '9'
  '9': [0, 0, 0, 0, 0, 0, 0, 0, 0];
};
```

String to Number

```ts
type StringToNumber<T extends string> = T extends
`${infer Res extends number}`
  ? Res
  : never;
```

```ts
// 123
StringToNumber<'123'>;
```

```ts
type ConcatStrings<
  T extends readonly string[],
  Accum extends string = ""
> = T extends [
  infer Fst extends string,
  ...infer TRest extends readonly string[]
]
  ? ConcatStrings<TRest, `${Accum}${Fst}`>
  : Accum;
```

```ts
// '123'
ConcatStrings<['1', '2', '3']>;
```

No Carry Calculator

```
type A = 123
type B = 456
type StrA = `${A}` // "123"
type StrB = `${B}` // "456"
type TupleA = StrToTuple<StrA> // ["1","2","3"]
type TupleB = StrToTuple<StrB> // ["4","5","6"]
type StrSum = SumTupleNoCarry<TupleA, TupleB> // "579"
type NumSum = StringToNumber<StrSum> // 579 🎉
```

CKEditor

Full Calculator
& much more!

https://github.com/Simply007/ts-type-arithmetic